

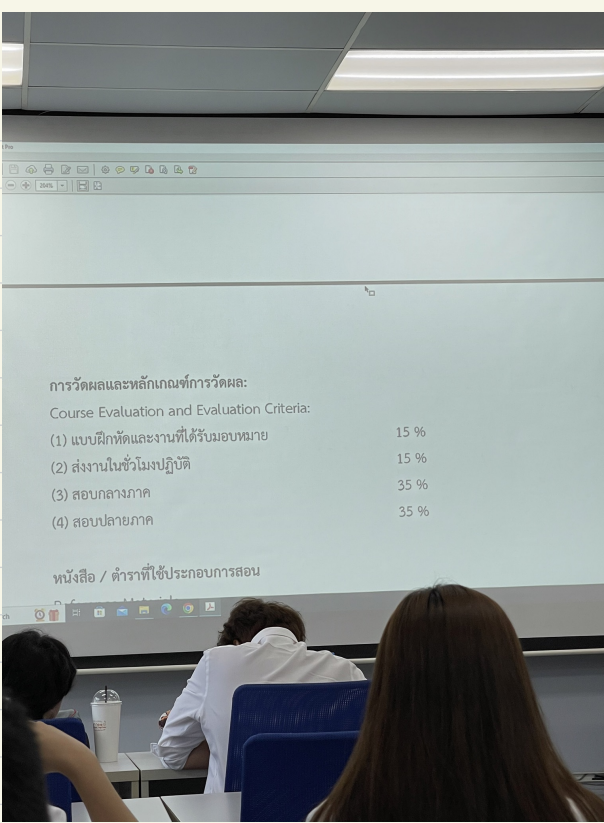
Data Struct

☺

Oh My...

sec 1

สอบ 70 ไร่



ทำไมต้องเรียน

เพื่อ นำไปใช้ เพิ่มประสิทธิภาพ ในการทำงาน โปรแกรม

- ฟังก์ชัน รวดเร็ว
- ประหยัดทรัพยากร

ประเด็นที่ต้องพิจารณา

- โครงสร้างข้อมูล
- อัลกอริทึม

ข้อมูลเชิงเดี่ยว (Atomic data)

เช่น ตัวเลข 1234

ข้อมูลประกอบ (Composite data)

เช่น หมายเลขโทรศัพท์

เลข หลักศึกษา

เลข บัตรประชาชน

3/2/67

Algorithms

คือ ขั้นตอนวิธีที่ใช้ในการแก้ปัญหา ซึ่งสามารถนำมาได้หลายรูปแบบ

- รหัสเทียม (Pseudo code)
- แผนผัง (Flowchart)
- โปรแกรม (Program)

การวิเคราะห์ อัลกอริทึม

เป้าหมายหนึ่งๆ สามารถถูกแก้ได้ด้วยหลายวิธี เพื่อเลือกรหัสที่ดีที่สุด เราต้องสามารถเปรียบเทียบประสิทธิภาพได้

การวิเคราะห์ อัลกอริทึม ปัญหา 1.1

Time Complexity เวลาที่ใช้ในการรันโปรแกรม ~~***~~

Space Complexity พื้นที่หน่วยความจำที่ใช้ในการรันโปรแกรม

ประสิทธิภาพของอัลกอริทึม

$f(n) = \text{efficiency}$ → จำนวนข้อมูล

ถ้าฟังก์ชันเป็น Linear (no loops)

Efficiency = จำนวนตัวแปร ที่ทำงานทั้งหมด

Ex. Algorithm A:

app-code

app-code

app-code

app-code

$f(n) = 4$

LINEAR LOOPS

- มีการเพิ่ม/ลดค่าภายในลูปอย่างคงที่

```
for (i=0; i<1000; i++)
    application code
end for
```

Iterations : 1000

```
for (i=0; i<1000; i+=2)
    application code
end for
```

Iterations : 500

```
for (i=0; i<n; i++)
    application code
end for
```

$f(n) = n$

```
for (i=0; i<n; i+=2)
    application code
end for
```

$f(n) = n/2$

LOGARITHMIC LOOPS

- เพิ่ม/ลดค่าภายในลูปเป็นจำนวนเท่าตัวด้วยการคูณหรือหาร

```
for (i=1; i<1000; i*=2)
    application code
end for
```

```
for (i=1000; i>1; i/=2)
    application code
end for
```

Iterations : 10 รอบ $\rightarrow \log_2 1000$

```
for (i=1; i<n; i*=2)
    application code
end for
```

```
for (i=n; i>1; i/=2)
    application code
end for
```

$f(n) = \log_2 n$

Nested Loops

- Linear Logarithmic
- Quadratic
- Dependent Quadratic

```
for (i=0; i<10; i++)
    for (j=1; j<10; j*=2)
        application code
    end for
end for
```

Iterations : $10 \log_2 10$; $f(n) = n \log_2 n$

```
for (i=0; i<10; i++)
    for (j=0; j<10; j++)
        application code
    end for
end for
```

Iterations : 100 ; $f(n) = n^2$

```
for (i=1; i<=10; i++)
    for (j=i; j<=10; j++)
        application code
    end for
end for
```

Iterations : 55 ; $f(n) = \frac{n}{2}(n+1)$

Ex. $\left[\begin{array}{l} \text{for}(i=0; i < 20; i++) \\ \quad \text{for}(j=1; j < 20; j*=2) \\ \quad \quad \text{app-code} \\ \quad \text{end for} \\ \text{end for} \end{array} \right] \log_2 n$

Iterations: $20 \log_2 20$ $f(n) = n \log_2 n$

Ex. $\left[\begin{array}{l} \text{for}(i=0; i < 15; i++) \\ \quad \text{for}(j=0; j < 15; j++) \\ \quad \quad \text{app-code} \\ \quad \text{end for} \\ \text{end for} \end{array} \right] n$

Iterations: 225 $f(n) = n^2$

Ex. $\left[\begin{array}{l} \text{for}(i=0; i < 20; i++) \\ \quad \text{for}(j=0; j < i; j++) \\ \quad \quad \text{app-code} \\ \quad \text{end for} \\ \text{end for} \end{array} \right] n$

Iterations: 210 $f(n) = \frac{n}{2}(n+1)$

Ex. $\left[\begin{array}{l} \text{for}(i=0; i < 10; i+=2) \\ \quad \text{for}(j=0; j < 10; j++) \\ \quad \quad \text{app-code} \\ \quad \text{end for} \\ \text{end for} \end{array} \right] n$

Iterations: 50 $f(n) = \frac{n^2}{2}$

Big-O Notation

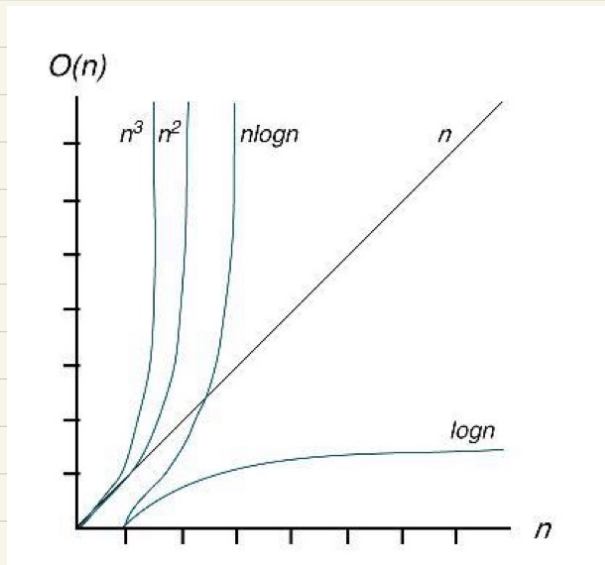
การจัดประสิทธิภาพนั้นขึ้นอยู่กับขนาดมาก และสนใจค่าประมาณ มากกว่า ซึ่ง เป็นการพิจารณาจาก อัตราการเติบโตของฟังก์ชัน $\rightarrow$ ดูจาก ค่าที่มีผลกระทบบมากที่สุดค่าเดียว

พิจารณา ว่า ลวดคอมไพ์มี สัมประสิทธิ์เป็น 1

สนใจเฉพาะ เทอมที่ใหญ่ที่สุด

$\log n$ n $n \log n$ n^2 n^3 ... n^k 2^n $n!$

น้อย —————→ มาก



Ex. `def printname(x):`
 `print(x + "Hello!")`

`print("P Chong")`

$O(1)$? 시간

$O(1)$? 일정한

Ex. `Algo A(x):`

`if x == 2:`

`app-code` 1

`else:`

`for(i=0; i<x; i++)`

`app-code`

`app-code`

`fn():`

$O(1)$? 시간

$O(n)$? 일정한

BIG-O ANALYSIS EXAMPLE

ขนาดของ Matrix

$n=3$

- Add Square Matrices

4	2	1
0	-3	4
5	6	2

 $+$

6	1	7
3	2	-1
4	6	2

 $=$

10	3	8
3	-1	3
9	12	4

8/2/67

Searching

เราสามารถใช้อัลกอริทึมมาช่วยประมวลผลงานทั่ว ๆ ไป
ผู้ใช้งานสามารถจัดการข้อมูล ที่เราเพิ่ม, แก้ไข หรือลบข้อมูลที่เราได้ อย่างรวดเร็ว
ถ้าเป็นของเร อัดอวกาศที่ เร่งในกรณีนี้ ข้อมูลได้ อย่างรวดเร็ว

Basic Search Algorithm

Sequential Search

เป็นการค้นหาข้อมูลในลักษณะ เรียงตามลำดับ โดยไล่ไปเรื่อย ๆ ให้พบ ซึ่งถ้าเจอข้อมูลที่ต้องการ

ตัวที่สอง ไล่ไปเรื่อยๆ จนกว่าจะพบข้อมูลที่ต้องการ

วิธีการประสิทธิภาพของกรณีนี้ พิจารณาแบ่งเป็น 3 กรณี

กรณีดีที่สุด (Best Case) 100 ครั้งแรก

กรณีแย่ที่สุด (Worst Case) 100 ตัวสุดท้าย หรือ 1000 ครั้ง

กรณีเฉลี่ย (Average Case) นำทุกตัวมาบวกแล้วหารค่าเฉลี่ย

- กรณีดีที่สุด สามารถค้นพบข้อมูลที่ต้องการได้ทันที (พบข้อมูลตั้งแต่ครั้งแรก)
 - ทำการเปรียบเทียบแค่ 1 ครั้ง
- กรณีแย่มากที่สุด ต้องทำการเปรียบเทียบข้อมูลไปจนถึงตัวสุดท้ายจึงจะพบ
 - ทำการเปรียบเทียบ N ครั้ง (มีข้อมูลทั้งหมด N ข้อมูล)
- กรณีเฉลี่ย หาโดยนำผลรวมของการเปรียบเทียบไปยังข้อมูลแต่ละตัว แล้วหารด้วยจำนวนข้อมูล
 - ทำการเปรียบเทียบเฉลี่ย $(1+2+\dots+N)/N = (N+1)/2$ ครั้ง

The Big-O Notation

Best case

เปรียบเทียบ 1 ครั้ง

$O(1)$

Average case

เปรียบเทียบเฉลี่ย $(N+1)/2$

$O(N)$

Worst Case

เปรียบเทียบ N ครั้ง

$O(N)$

Binary Search

การค้นหาแบบ sequential ง่าย แต่เสียเวลามาก

การค้นหาที่มีประสิทธิภาพมากขึ้น ได้แก่ การค้นหาแบบ Binary Search

เป็นการค้นหาที่ใช้ข้อมูลที่มีการเรียงลำดับ แล้วเท่านั้น

ทำตารางเปรียบเทียบข้อมูล ที่จุดค่า 1 แถว กว้างลบ ของชุดข้อมูล หัวเขด
ถ้ายังไม่พบก็ไป เปรียบ เติบโตไปเรื่อยๆ

ค่า 1 แถว กว้างกลาง : $\lfloor (begin + end) \rfloor$

Big-O of Binary Search

Best Case

เปรียบเทียบ 1 ครั้ง ข้อมูลอยู่กึ่งกลางพอดี

$O(1)$

Average Case

เปรียบเทียบเฉลี่ย $(2^0 \times 1 + 2^1 \times 2 + 2^2 \times 3 + \dots + 2^m \times (m+1)) / N$

$O(\log_2 N)$

Worst Case

เปรียบเทียบตลอด ข้อมูลอยู่ปลายสุดของ array จนแล้วค่อยพิจารณา

$O(\log_2 N)$

Hashing

ต้องการให้การค้นหาข้อมูลมีประสิทธิภาพเป็น $O(1)$ นั่นคือ สามารถเข้าถึงข้อมูลได้โดยตรง
ข้อมูลที่เข้าไว้ที่ตำแหน่งใด ๆ จะเรียกว่า คีย์ (key)
ใช้ Hash Function มาคำนวณคีย์นั้นได้กับข้อมูล ซึ่งเรียกว่า ตารางแฮช

Collision

การชนกันของข้อมูล นั่นคือ คีย์ไม่เหมือนกันแต่แฮชได้ค่าเดียวกัน ตารางแฮช

วิธีแก้ปัญหาคollision

- ใช้ Good Hash Function ให้ดี
- ทำ Rehashing

Collision Resolution

- Linear Probing
- Buckets
- Chaining (Linked List)

Ex. 1 61 72 14 16 17 23

for Binary Search in 17 เป็นบัพทิสหะนาคัก

1 14 16 17 23 61 72

mid > l
< r

Ex. 1 6 72 14 16 17 23

for Sequential in 17 เป็นบัพทิสหะนาคัก 8

Linear Probing คือ การค้นหาแบบง่าย ๆ ครับ

- ในกรณีที่ต้องการค้นหาข้อมูล
 - นำคีย์ไปผ่าน Hash Function ถ้าเจอข้อมูลที่มีคีย์ตรงกันแสดงว่าหาเจอ
 - ถ้าไม่ใช่คีย์ที่ต้องการ ให้ไล่ไปตำแหน่งถัดไปเรื่อยๆ จนกว่าจะเจอ
 - แต่ถ้าเจอช่องว่าง แสดงว่า ไม่มีข้อมูลนั้น
- เมื่อเกิดการชนกัน จะไล่ไปตำแหน่งถัดไป นั่นก็คือ การทำ Rehashing ด้วยฟังก์ชัน
 - (ตำแหน่งที่แฮชได้ + ค่าคงที่) % ขนาดอาร์เรย์
- การ Hashing เป็นวิธีที่ดี แต่เมื่อเกิดการชนกันมากขึ้น จะทำให้ประสิทธิภาพการค้นหาข้อมูลแยกลง

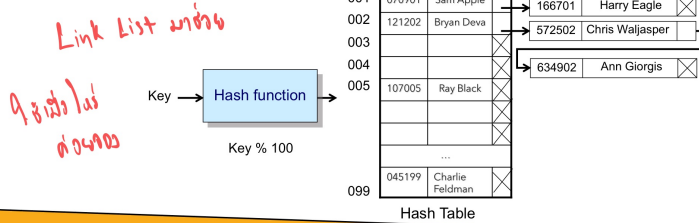
BUCKETS

- คีย์ที่ผ่านการแฮชแล้วได้ค่าเหมือนกัน สามารถเก็บอยู่ในตำแหน่งเดียวกันได้ ซึ่งมีการกำหนดจำนวนของคีย์ที่ตำแหน่งเดียวกันชัดเจน
- เช่น กำหนดให้มีคีย์ที่ตำแหน่งเดียวกันไม่เกิน 3 คีย์
- เก็บข้อมูลตารางแฮชเป็นแบบอาร์เรย์ 2 มิติ
- ปัญหา หากมีคีย์ที่แฮชแล้วจะอยู่ตำแหน่งเดียวกันมากกว่าที่กำหนด ก็จะมีการชนกันอีก

Chaining trong Linked List

CHAINING

- คีย์ที่ผ่านการแฮชแล้วได้ค่าเหมือนกัน สามารถเก็บอยู่ในตำแหน่งเดียวกันได้ โดยไม่จำกัดจำนวน
- เก็บข้อมูลตารางแฮชเป็นแบบลิงค์ลิสต์



13/3/67 Sorting

พิจารณาวิธีขจัดมูลจากน้องหมา

25 Algorithms

Insertion Sort

Selection Sort

Bubble Sort

Merge Sort

Quick Sort

Insertion Sort

လမ်းစဉ်မှာ ၂ မြေ နဲ့ ၀

๑. ลักษณะข้อบกพร่องที่ ถูก 15 ข้อ

2. กำหนดข้อที่ ข้างในวงเล็บ

$$f(n) = (n-1)(n/2)$$

$O(n^2)$

Ex. 729 13 16 27 11 101 54

1	13	729					
2	13	16	729				
2	13	16	27	729			
4	11	13	16	27	729		
2	11	13	16	27	101	729	
3	11	13	16	27	54	101	729
14							

Selection Sort $f(n) = \frac{n(n-1)}{2}$ $O(n^2)$

Ex. 729 13 16 27 11 101 54

21	{	6	11	13	16	27	729	101	54
		5	11	13					
		4	11	13	16				
		3	11	13	16	27			
		2	11	13	16	27	54	101	729
		1	11	13	16	27	54	101	729

Bubble Sort

เริ่มพิจารณาไปเรื่อยๆ จากข้อมูลที่มีอยู่ Unsorted Sublist

$$f(n) = \frac{n(n-1)}{2} \quad O(n^2)$$

Ex. 729 13 16 27 11 101 54

21	6	11	729	13	16	27	54	101
	5	11	13	729	16	27	54	101
	4	11	13	16	729	27	54	101
	3	11	13	16	27	729	54	101
	2	11	13	16	27	54	729	101
	1	11	13	16	27	54	101	729

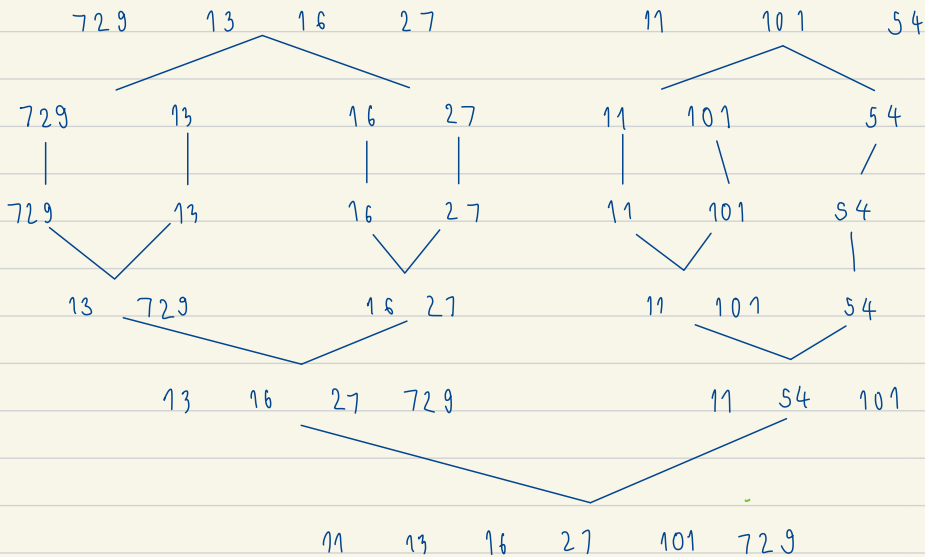
Merge Sort

แบ่งลิสต์ย่อยออกที่ 2 ส่วน (Sublist) ให้สั้น จนกระทั่งได้ 1

รวมกลับ

$$f(n) = n \log_2 n - (n-1) \quad O(n \log_2 n)$$

Ex 729 13 16 27 11 101 54



14

Quick Sort $O(n \log_2 n)$ ถ้า Pivot ดี $O(n^2)$ Pivot ไม่ดี

ใช้ ค่า Pivot แบ่งลิสต์ออกเป็น 2 ชุด แล้วลิสต์ที่ถูกแบ่งไว้จะเรียงตัวจนครบค่ากับ Pivot เป็นค่าที่แน่นอน และลิสต์ที่เหลือก็ทำการแบ่ง

Ex. 729 13 16 27 11 101 54

11 13 16 27 54 101 729

Greedy Algorithms

อัลกอริทึมที่ทำการแก้ปัญหาโดยการเลือกทำสิ่งที่ได้ผลลัพธ์ที่ดีที่สุดในขณะนั้น
ไม่รับประกันว่าผลลัพธ์จาก greedy จะเป็นผลลัพธ์ที่ดีที่สุด (optimal solution)
เป็นที่ใช้ Greedy

- Coin exchange
- Set-covering
- Classroom scheduling
- Traveling salesman
- Knapsack

Ex. ต้องการหอบรรจุถุง 45 บาท โดยใส่เหรียญ 10, 5, 2, 1 บาท

$$10 \times 4 = 40$$

$$2 \times 1 = 2$$

$$1 \times 1 = 1$$

$$= 43 = 6 \text{ เหรียญ}$$

Ex. มีกระเป๋าที่รับได้หนักได้ 3 kg มีสิ่งของดังนี้ 1. Laptop 2 kg น้ำหนัก 5 k
2. หนังสือ 1 kg น้ำหนัก 1 k 3. เสื้อผ้า 1 kg 2 k 4. tablet 1 kg น้ำหนัก 4 k
ถ้าเลือกแบบของจากสิ่งที่น้ำหนักมากที่สุด

$$\text{Laptop} + \text{tablet} = 9 \text{ k}$$

วิธีที่ดีเป็น optimal หรือไม่ (ของหนักไปเลย) X

Recursion

Ex. AlgorA(x, y):

if $x == 5$:

return x

elif $y < x$:

return $y + 2$

else:

return $x + \text{AlgorA}(x+1, y-2)$

AlgorA(7, ⁹10) = ? 16

AlgorA(5, 10) = ? 5

AlgorA(7, 1) = ? 3

Ex. Algor B(x):

```
if x == 5:  
    print("New Sud Lor")  
elif x == 7:  
    print("Arm Sud Tae")  
else:  
    print("This Number is" + x)  
    Algor B(x+3)  
    print("SOM !!!")
```

Algor B(2) = ?

Algor B(1) = ?

Divide and Conquer Algorithms

หลักการ แบ่งออก แล้ว รวม

เป็นขั้นตอนการแก้ปัญหา โดยนำปัญหามาแบ่งออกเป็นส่วนย่อย

Divide แยกปัญหามา

Conquer แก้ไข Recursion

Combine

* binary search เป็นขั้นตอนการ Divide and Conquer อีก*

Merge เป็นขั้นตอนการ Divide and Conquer อีก

$$\begin{array}{l} 4^{150} \text{ mod } 3 = ? \\ 4^{125} \quad \quad \quad = \\ 4^{62} \quad \quad \quad = \\ 4^{31} \quad \quad \quad = \\ 4^{15} \quad \quad \quad = \\ 4^7 \quad \quad \quad = \\ 4^3 \quad \quad \quad = \\ 4 \quad \quad \quad = \\ 4^0 \quad \quad \quad = \end{array}$$

$$\begin{array}{l} (4^{125} \text{ mod } 3)^2 \text{ mod } 3 = 1 \\ 4 \times (4^{62} \text{ mod } 3)^2 \text{ mod } 3 = 1 \\ (4^{31} \text{ mod } 3)^2 \text{ mod } 3 = 1 \\ 4 \times (4^{15} \text{ mod } 3)^2 \text{ mod } 3 = 1 \\ 4 \times (4^7 \text{ mod } 3)^2 \text{ mod } 3 = 1 \\ 4 \times (4^3 \text{ mod } 3)^2 \text{ mod } 3 = 1 \\ 4 \times (4 \text{ mod } 3)^2 \text{ mod } 3 = 1 \\ (4 \text{ mod } 3) \text{ mod } 3 = 1 \end{array}$$

Dynamic Programming

หลักการสำคัญ: 1. ศึกษารายละเอียดโครงการให้ถี่ถ้วนก่อนลงมือทำ 2. จัดทำแผนการทำงาน 3. ให้ความสำคัญกับความปลอดภัย 4. ให้ความสำคัญกับคุณภาพงาน 5. ให้ความสำคัญกับเวลา 6. ให้ความสำคัญกับต้นทุน 7. ให้ความสำคัญกับสิ่งแวดล้อม 8. ให้ความสำคัญกับสังคม 9. ให้ความสำคัญกับจริยธรรม 10. ให้ความสำคัญกับนวัตกรรม

ประวัติปัญหา: การทำสัญญาซื้อขาย ที่ 98 ไปรษณีย์พาณิชย์

ห้องปฎิบัติการมาตรฐานวัดไฟฟ้า → ช่างไฟฟ้าสมัคร ห้องช่างไฟฟ้าสมัคร

การหาค่า Optimize นั้นมี

Ex. สินค้าที่ขายได้วันละ 3 kg สินค้าตัวที่ 1, Laptop 2 kg ยานี่ 5k
2. สินค้า 1 kg ยานี่ 1k 3. สินค้า 1 kg 2k 4. tablet 1 kg ยานี่ 4k

1 kg 2 kg 3 kg 4k ยานี่ 2k

lap top		5 k	5 k
watch	1 k	5 k	5 k + 1 k 6 k
ring	2 k	5 k	5 k + 2 k 7 k
tablet	4 k	4 k + 2 k 6 k	5 k + 4 k 9 k

Laptop + Book
|| + clothes

Longest Sub String

	B	e	t	t	e	r
l	0	0	0	0	0	0
e	0	1	0	0	1	0
t	0	0	2	1	0	0
t	0	0	1	3	0	0
e	0	1	0	0	4	0
r	0	0	0	0	0	5

	f	i	r	s	t
f	1	0	0	0	0
r	0	0	1	0	0
o	0	0	0	0	0
s	0	0	0	1	0
t	0	0	0	0	2

Longest Sub sequence

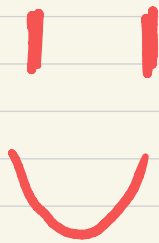
	B	e	t	t	e	r
l	0	0	0	0	0	0
e	0	1	1	1	1	1
t	0	1	2	2	2	2
t	0	1	2	3	3	3
e	0	1	2	3	4	4
r	0	1	2	3	4	5

	f	i	r	s	t
f	1	1	1	1	1
r	1	1	2	2	2
o	1	1	2	2	2
s	1	1	2	3	3
t	1	1	2	3	4

```
for (i = 1; i <= 10; i++)  
    for (j = 0; j <= i; j++)
```

บอกให้สอบได้

ออกตามที่อ่าน



By TaiKie